

Computer Programming Summer Interest Group

Session 22 / Testing

Hebrews 12:7-11 (ESV)

It is for discipline that you have to endure. God is treating you as sons. For what son is there whom his father does not discipline? If you are left without discipline, in which all have participated, then you are illegitimate children and not sons. Besides this, we have had earthly fathers who disciplined us and we respected them. Shall we not much more be subject to the Father of spirits and live? For they disciplined us for a short time as it seemed best to them, but he disciplines us for our good, that we may share his holiness. For the moment all discipline seems painful rather than pleasant, but later it yields the peaceful fruit of righteousness to those who have been trained by it.

Context for Virtual Environments

Hebrews 12:7-11 reveals God's desire for us to work according to His design

- He disciplines us so we can stand when and where He wants us to be His witnesses

Testing software performs a similar function, helping the developer gain assurance the program will function according to the intention of the developer.

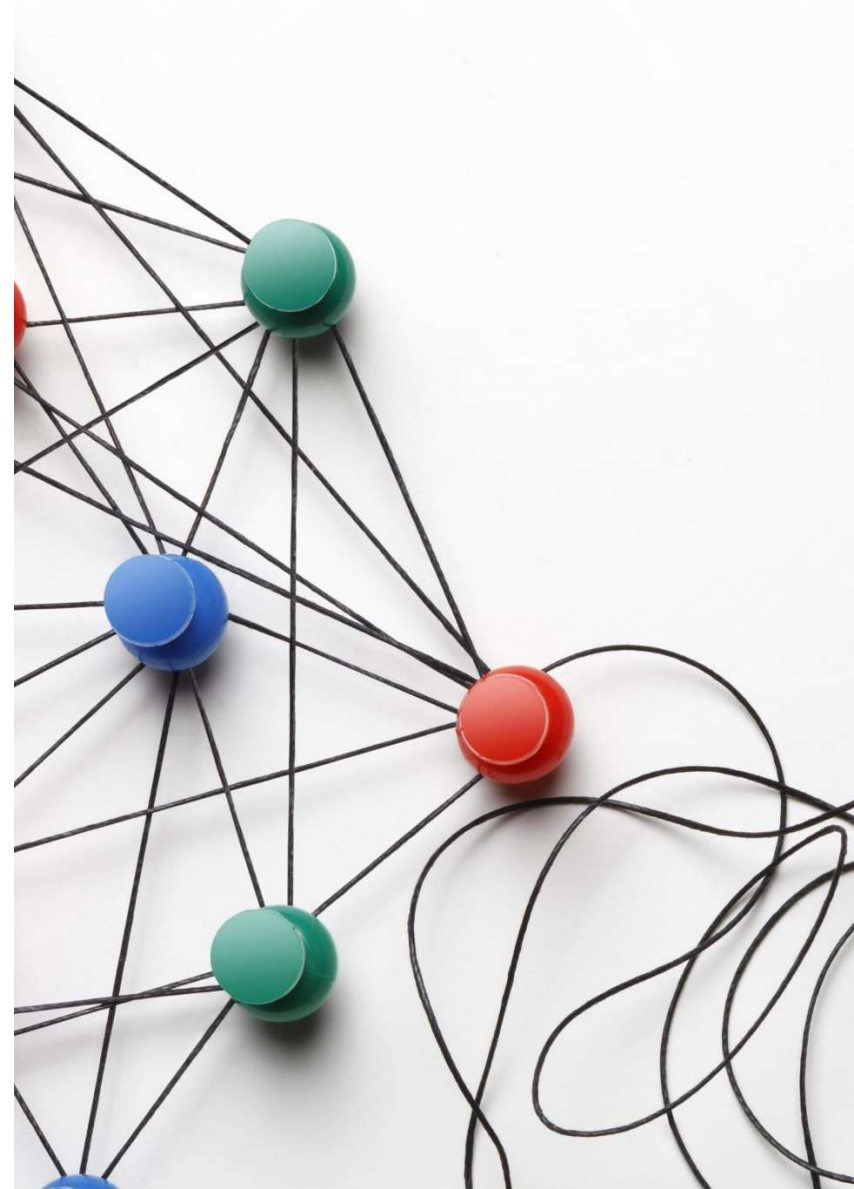
Unit testing is an activity which is very helpful in ensuring components of a program function as intended.

Test driven development is a methodology which can be used to aid in developing software which meets the developer's expectations.

The Python environment provides tools to aid in performing unit tests to provide some assurance software will perform as expected.

Objectives

- By the end of this session, you will be able to:
 - Explain the principles of Test-Driven Development (TDD).
 - Write unit tests using unittest.
 - Write unit tests using pytest.
 - Determine the code coverage when running unit tests.
 - Apply the Red–Green–Refactor cycle.



Unit Testing

- The purpose of unit tests is to ensure code functions as intended
- Increases assurance of proper operation to enable refactoring to address code quality or performance issues without changing outcomes.
- Unit tests should test interfaces intended for other components to use
- Unit tests should test the behavior of the code, and avoid testing that it conforms to some specific design or implementation
- The collection of unit tests should be designed to completely test all the code to be tested
 - All branches should be tests
 - This is called coverage; ideally tests should provide 100% coverage
- Code should be implemented with testability in mind

Code to test

- This code will be subject to testing in the remainder of this presentation
- Fibonacci
 - Takes a single argument, n and returns the nth Fibonacci number.
 - The sequence of Fibonacci numbers is recursively defined on the non-negative integers by the following:
$$fib(n) = \begin{cases} n = 0 & 0 \\ n = 1 & 1 \\ n > 1 & fib(n - 1) + fib(n - 2) \end{cases}$$
- Factorial
 - Takes a single argument, n and returns n! (n factorial) which is the product of the positive integers less than or equal to n.
 - By convention 0! = 1
- In the following pages we will write unit tests to ensure these functions operate correctly, including when given bad inputs.

```
def fibonacci(n: int) -> int:
    if n < 0:
        raise ValueError("Expected n>=0")
    if n < 2:
        return n
    a, b = 0, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b
```

```
def factorial(n: int) -> int:
    if n < 0:
        raise ValueError("Expected n>=0")
    if n == 0:
        return 1
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

The unittest package

- Python has a built-in unit testing package called `unittest`
- Does not require additional package installation, but it does require an import.
- Tests are included as methods in a class derived from `unittest.TestCase`.
- We need to import the `unittest` module and the module to test
- Two functions at the top can be used to generate test cases
 - The first element of each tuple is the input and the second is the expected output
- The `TestSampleFunctions` class contains helper functions and test functions
 - The `unittest` library treats all methods starting with the word “test” as individual test method
 - Any others can be used as helper functions, but are not considered tests
- To run the test use:
 - `python -m unittest`
 - This will run any tests found in files with names starting with the word “test”
- The `run_test_cases` helper function runs the test cases provided to it on the function given
- The `run_bad_value` helper function expects the function given to raise a value error when given the value

```
import unittest
from sample_22_01 import fibonacci, factorial

def fibonacci_test_cases() -> list[tuple[int, int]]:
    return [(0, 0), (1, 1), (5, 5), (10, 55)]

def factorial_test_cases() -> list[tuple[int, int]]:
    return [(0, 1), (1, 1), (5, 120), (10, 3628800)]

class TestSampleFunctions(unittest.TestCase):
    def run_test_cases(self, func, test_cases):
        for n, expected in test_cases:
            with self.subTest(n=n, expected=expected):
                self.assertEqual(func(n), expected)

    def run_bad_value(self, func, value):
        with self.assertRaises(ValueError):
            func(value)

    def test_fib(self):
        self.run_test_cases(fibonacci, fibonacci_test_cases())

    def test_fib_negative(self):
        self.run_bad_value(fibonacci, -1)

    def test_factorial(self):
        self.run_test_cases(factorial, factorial_test_cases())

    def test_factorial_negative(self):
        self.run_bad_value(factorial, -1)
```

The pytest package

- When using pytest, we need to import the pytest package as well as the package containing the code to test.
- Here, we do not use a class, but standalone functions with names starting with the word “test”
 - Functions not starting with the word test are not included in the test results but may be referenced by test functions as helpers.
- Again, we have functions that return tuples containing an input value and expected output value for each fibonacci numbers and factorials.
- When running tests, we can use the `@pytest.mark.parametrize` decorator to run through each case to ensure the function provides the correct output for each input.
 - Simplifies the implementation for this testing pattern
- We can also see how to test a function that is expected to raise an exception when given an invalid value in `test_fibonacci_negative` and `test_factorial_negative`.
- From the command line, run pytest by using:
 - `pytest`

```
import pytest
from sample_22_01 import fibonacci, factorial

def fibonacci_test_cases() -> list[tuple[int, int]]:
    return [(0, 0), (1, 1), (5, 5), (10, 55)]

def factorial_test_cases() -> list[tuple[int, int]]:
    return [(0, 1), (1, 1), (5, 120), (10, 3628800)]

@pytest.mark.parametrize("n, expected",
    fibonacci_test_cases())
def test_fibonacci(n, expected):
    assert fibonacci(n) == expected

def test_fibonacci_negative():
    with pytest.raises(ValueError):
        fibonacci(-1)

@pytest.mark.parametrize("n, expected",
    factorial_test_cases())
def test_factorial(n, expected):
    assert factorial(n) == expected

def test_factorial_negative():
    with pytest.raises(ValueError):
        factorial(-1)
```


Unit test code coverage

Code coverage refers to the lines of code or conditional branches executed during a run of an application

The coverage tool is used in conjunction with a unit testing tool to determine which lines a test suite actually executes

Tests should be crafted to execute all lines of code and all branching conditions (e.g., if statement branches)

- When using unittest:
 - `coverage run -m unittest`
- When using pytest:
 - `coverage run -m pytest`
- To see coverage report results:
 - `coverage report --show-missing`

Test Driven Development (TDD)

Traditionally developers write code and then write tests to test the code

Test Driven Development (TDD) is the process of writing tests first and then writing code to pass the test

This ensures the code written is testable and subject to existing tests

Code coverage is generally better since developer only writes code necessary to pass the test

Code can be refactored and functionality retained as long as tests pass

+

•

○

Test Driven Development Process

1. **PLANNING: List scenarios for the new feature**
 - List all characteristics of the new feature to include
2. **RED: Write a test for an item on the list**
 - Write an automated test that *would* pass if the variant in the new behavior is met.
 - Run all tests. The new test should *fail* – for *expected* reasons
 - This shows new code is needed for the desired feature.
 - It validates that the test harness is working correctly.
 - It rules out the possibility that the new test is flawed and will always pass.
3. **GREEN: Write the simplest code that passes the new test**
 - Inelegant code and hard coding is acceptable.
 - No code should be added beyond the tested functionality.
 - All tests should now pass
 - Make minimal changes necessary to fix code base if any tests fail
4. **REFACTOR: Refactor as needed while ensuring all tests continue to pass**
 - Code is refactored for readability and maintainability.
 - Removing duplicate code
 - Making names self-documenting
 - Splitting methods into smaller pieces
 - Re-arranging inheritance hierarchies
 - Run test suite after each refactor ensures that no existing functionality is broken.
5. **REPEAT**
 - Repeat the process, starting at step 2, with each test on the list until all tests are implemented and passing.

Source: [Test Driven Development](https://en.wikipedia.org/wiki/Test_driven_development) (Wikipedia.org)



Activities

- A heap data structure is a list with the following property:
 - $\text{heap}[n] \leq \text{heap}[2*n]$ if the heap size $< 2*n$
 - $\text{heap}[n] \leq \text{heap}[2*n+1]$ if the heap size $< 2*n+1$
- Write both pytest and unittest modules to test that a function given a list of integers in arbitrary order, returns a heap which is a reordering of the original list.
 - That is, the function reorders a list of integer values to ensure the resulting list forms a heap
 - Create a collection of input and output lists to serve as tests for the unit test cases
- Write a module that contains a function which when given a list will pass the tests
- Note: There are built in functions in python which “heapify” a list of values; your final solution should not use any of those functions; you should write the code to perform the reordering.